2019 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)
Macau, China, November 4-8, 2019

Dynamic Input for Deep Reinforcement Learning in Autonomous Driving

Maria Huegle[1,*], Gabriel Kalweit[1,*], Branka Mirchevska[2], Moritz Werling[2], Joschka Boedecker[1,3]

***Abstract*— In many real-world decision making problems, reaching an optimal decision requires taking into account a variable number of objects around the agent. Autonomous driving is a domain in which this is especially relevant, since the number of cars surrounding the agent varies considerably over time and affects the optimal action to be taken. Classical methods that process object lists can deal with this requirement. However, to take advantage of recent high-performing methods based on deep reinforcement learning in modular pipelines, special architectures are necessary. For these, a number of options exist, but a thorough comparison of the different possibilities is missing. In this paper, we elaborate limitations of fully-connected neural networks and other established approaches like convolutional and recurrent neural networks in the context of reinforcement learning problems that have to deal with variable sized inputs. We employ the structure of Deep Sets in off-policy reinforcement learning for high-level decision making, highlight their capabilities to alleviate these limitations, and show that Deep Sets not only yield the best overall performance but also offer better generalization to unseen situations than the other approaches.**

I. Introduction

Many autonomous driving systems are built upon a modular pipeline consisting of perception, localization, mapping, high-level decision making and motion planning. The perception component extracts a list of surrounding objects and traffic participants like vehicles, pedestrians and bicycles. The number of objects that are relevant for the later decision making process can be highly dynamic, in both highway and urban scenarios. Classical rule-based decision-making systems can process these lists directly, but are limited due to their fragility in light of ambiguous and noisy sensor data. Deep Reinforcement Learning (DRL) methods offer an attractive alternative for learning decision policies from data automatically and have shown great potential in a number of domains [1], [2], [3], [4]. Promising results were also shown for learning driving policies from raw sensor data [5]. However, end-to-end methods can suffer from a lack of interpretability and can be difficult to train. These issues can be alleviated by learning from low-dimensional state features extracted by a perception module. To take advantage of DRL methods in scenarios with dynamic input lengths, though, special architectural components are necessary as part of the learning system. Several neural network based options allow processing of variable-sized input, as detailed below. However, it is not clear which of these is superior in context of a DRL algorithm, enabling high performance and generalization to various scenarios.

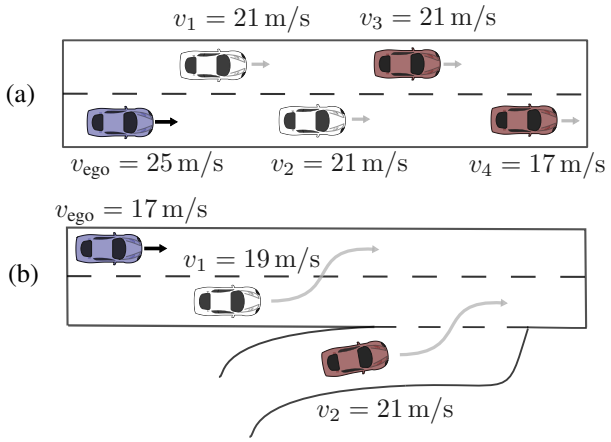


Fig. 1. Limitations of fixed input representations consisting of (a) the 2-nearest surrounding vehicles, where vehicles 3 and 4 are not visible (b) a relational grid with $\Delta_{\text{lateral}} = 1$ considered side lanes. Cars considered by the input representation are shown in white, invisible cars in red and the agent in blue.

Prior research mostly relied on fixed sized inputs [6], [7], [8], [9] or occupancy grid representations [10], [11] for value or policy estimation. However, fixed sized inputs limit the number of considered vehicles, for example to the n-nearest vehicles. Figure 1 (a) shows a scenario where this kind of representation is not enough to make the optimal lane change decision, since the agent does not represent the $n+1$ and $n+2$ closest cars even though they are within sensor range. A more advanced fixed sized representation [6], [7] considers a relational grid of Δ_{ahead} leaders and Δ_{behind} followers in Δ_{lateral} side lanes around the agent. As Figure 1 (b) shows, this can still be insufficient for optimal decision making, since the white car on the incoming $\Delta_{\text{lateral}} + 1$ right lane is not within the maximum of considered lanes, and therefore has no influence on the decision making process.

Using occupancy grids in combination with convolutional neural networks (CNN) imposes a trade-off between computational workload and expressiveness. Whilst smaller architectures acting on low-resolution grids as an input are efficient from a computational perspective, they may be too imprecise to represent the environment correctly. Whereas for high-resolution grids, most computations can be poten-

*Authors contributed equally.
[1]Dept. of Computer Science, University of Freiburg, Germany.
{hueglem,kalweitg,jboedeck}@cs.uni-freiburg.de
[2]BMWGroup, Unterschleissheim, Germany.
{Branka.Mirchevska,Moritz.Werling}@bmw.de
[3]Cluster of Excellence BrainLinks-BrainTools, Freiburg, Germany.

Authorized licensed use limited to: SHANGHAI UNIVERSITY. Downloaded on October 22,2026 at 07:28:47 UTC from IEEE Xplore. Restrictions apply.

tially redundant due to the sparsity of grid maps. In addition, a grid is limited to its initially defined size. Objects off the grid cannot be represented and are therefore not considered in decision making.

A third way to deal with variable input sequences are recurrent neural networks (RNNs). Even though the temporal context might be important for predictions w.r.t. a single or a group of objects, the order in which the objects are fed into the value function or policy estimator should not be relevant for a fixed time step – i.e., the input should be *permutation invariant*. Combined with an attention mechanism, a recurrent network can be used to create a set representation which is permutation invariant w.r.t. the input elements [12], which we subsequently call Set2Set. However, RNNs tend to be difficult to train [13], e.g. due to vanishing and exploding gradients, which can be a problem in highly stochastic environments.

Lastly, Deep Sets can be employed to process inputs of varying size [14]. The Deep Set architecture was used for point cloud classification and anomaly detection [14], as well as in a sim-to-real DQN-setting for a robot sorting task [15].

In this paper, we suggest to use Deep Sets for deep reinforcement learning as a flexible and permutation invariant architecture to handle a variable number of surrounding vehicles that influence the decision making process for lane-change maneuvers. We additionally propose to use the Set2Set architecture as both baseline and another new approach to deal with a variable input size in reinforcement learning. Our main contributions are the formalization of the DeepSet-Q and Set2Set-Q algorithms, and the extensive evaluation of DeepSet-Q, comparing it to various other approaches to handle dynamic inputs, and showing that Deep Sets outperform related approaches in the application of high-level decision making in autonomous lane changes, while generalizing better to unseen situations.

II. Method

In reinforcement learning, an agent acts in an environment by applying action $a_t \sim \pi$, following policy π, in state s_t, gets into some state $s_{t+1} \sim \mathcal{M}$, according to model $\mathcal{M}$, and receives a reward r_t in each discrete time step t. The agent has to adjust its policy π to maximize the discounted long-term return $R(s_t) = \sum_{t'>=t} \gamma^{t'-t} r_{t'}$, where $\gamma \in [0, 1]$ is the discount factor. We focus on the case, where we find the optimal policy based on model-free Q-learning [16]. The Q-function $Q^\pi(s_t, a_t) = \mathbf{E}_{a_{t'>t} \sim \pi}[R(s_t)|a_t]$ represents the value of an action a_t and following π thereafter. From the optimal action-value function Q^*, we can then easily extract the optimal policy by maximization.

A. DeepSet-Q

In the DeepSet-Q approach, we train a neural network $Q_{\mathcal{DS}}(\cdot, \cdot|\theta^{Q_{\mathcal{DS}}})$, parameterized by $\theta^{Q_{\mathcal{DS}}}$, to estimate the action-value function via DQN [1] for state

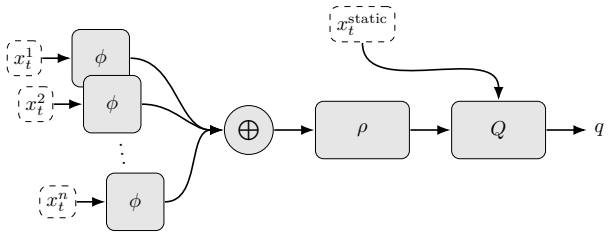


Fig. 2. Scheme of DeepSet-Q. The modules ϕ, ρ and Q are fully-connected neural networks and $\bigoplus$ is a permutation invariant pooling operation. Vectors x_t^j and x_t^{static} are dynamic and static feature representations at time step t, q is the action-value output.

$s_t = (X_t^{\text{dyn}}, x_t^{\text{static}})$ and action a_t. The state consists of a dynamic input $X_t^{\text{dyn}} = [x_t^1, .., x_t^{\text{seq len}}]^\top$ with a variable number[1] of vectors $x_t^j|_{0 \le j \le \text{seq len}}$ and a static input vector x_t^{static}.

The Q-network $Q_{\mathcal{DS}}$ consists of three main parts, (ϕ, ρ, Q). The input layers are built of two neural networks ϕ and ρ, which are components of the Deep Sets. The representation of the input set is computed by:

$$\Psi(X_t^{\text{dyn}}) = \rho\left(\sum_{x_t^j \in X_t^{\text{dyn}}} \phi(x_t^j)\right),$$

which makes the Q-function permutation invariant w.r.t. its input [14]. An overview of the Q-function is shown in Figure 2. Instead of taking the *sum*, other permutation invariant pooling functions, such as the *max*, can be used. Static feature representations x_t^{static} can be fed directly to the Q-module. The final Q-values are then given by $Q_{\mathcal{DS}}(s_t, a_t) = Q([\Psi(X_t^{\text{dyn}}), x_t^{\text{static}}], a_t)$, where $[\cdot, \cdot]$ denotes concatenation. The Q-function is trained on minibatches sampled from a replay buffer $\mathcal{R}$, which contains transitions collected by some policy $\hat{\pi}$. We then minimize the loss function:

$$L(\theta^{Q_{\mathcal{DS}}}) = \frac{1}{b} \sum_i \left(y_i - Q_{\mathcal{DS}}(s_i, a_i)\right)^2,$$

with targets $y_i = r_i + \gamma \max_a Q'_{\mathcal{DS}}(s_{i+1}, a)$, where $(s_i, a_i, s_{i+1}, r_i)|_{0 \le i \le b}$ is a randomly sampled minibatch from the replay buffer. The target network is a slowly updated copy of the Q-network. In every time step, the parameters of the target network $\theta^{Q'_{\mathcal{DS}}}$ are moved towards the parameters of the Q-network by step-length τ, i.e. $\theta^{Q'_{\mathcal{DS}}} \leftarrow \tau\theta^{Q_{\mathcal{DS}}} + (1-\tau)\theta^{Q'_{\mathcal{DS}}}$. For a detailed description, see Algorithm 1. To overcome the problem of overestimation, we further apply a variant of Double-Q-learning which is based on two independent network pairs, so as to use the minimum of the predictions for target calculation [17].

B. Set2Set-Q

Another approach to deal with a variable number of input elements is to replace $Q_{\mathcal{DS}}$ and target network $Q'_{\mathcal{DS}}$ by recurrent network architectures Q_{S2S} and Q'_{S2S}. Here, the input layer of the Q-network consists of the recurrent module

[1] In the application of autonomous driving, the sequence length is equal to the number of vehicles in sensor range.

Authorized licensed use limited to: SHANGHAI UNIVERSITY. Downloaded on October 22,2026 at 07:28:47 UTC from IEEE Xplore. Restrictions apply.

Algorithm 1: DeepSet-Q

initialize $Q_{\mathcal{DS}} = (\phi, \rho, Q)$ and $Q'_{\mathcal{DS}} = (\phi', \rho', Q')$
set replay buffer $\mathcal{R}$
for *optimization step o=1,2,...* **do**
  get minibatch $(s_i, a_i, (X^{\text{dyn}}_{i+1}, x^{\text{static}}_{i+1}), r_i)$ from $\mathcal{R}$
  foreach *transition* **do**
    foreach *object* x^j_{i+1} *in* X^{dyn}_{i+1} **do**
      $(\phi'_{i+1})^j = \phi'\left(x^j_{i+1}\right)$
    end
    $\rho'_{i+1} = \rho'\left(\sum_j (\phi'_{i+1})^j\right)$
    $y_i = r_i + \gamma \max_a Q'(\rho'_{i+1}, x^{\text{static}}_{i+1}, a)$
  end
  perform a gradient step on loss:

$$\frac{1}{b}\sum_i \left(Q_{\mathcal{DS}}(s_i, a_i) - y_i\right)^2$$

  update target network by:

$$\theta^{Q'_{\mathcal{DS}}} \leftarrow \tau\theta^{Q_{\mathcal{DS}}} + (1-\tau)\theta^{Q'_{\mathcal{DS}}}$$

end

Algorithm 2: Set2Set-Q

initialize $Q_{S2S} = (\text{LSTM}, Q)$ and $Q'_{S2S} = (\text{LSTM}', Q')$
set replay buffer $\mathcal{R}$
for *optimization step o=1,2,...* **do**
  get minibatch $(s_i, a_i, (X^{\text{dyn}}_{i+1}, x^{\text{static}}_{i+1}), r_i)$ from $\mathcal{R}$
  foreach *transition* **do**
    reset *hidden* state of LSTM′ and q^*_0
    for *iteration k=1,2,..., K* **do**
      $q_k = \text{LSTM}'(q^*_{k-1})$
      foreach *object* x^j_{i+1} *in* X^{dyn}_{i+1} **do**
        $e_{j,k} = x^j_{i+1} \cdot q_k$
      end
      $\alpha_{j,k} = \frac{exp(e_{j,k})}{\sum_m exp(e_{m,k})}$
      $\beta_k = \sum_j \alpha_{j,k} x^j_{i+1}$
      $q^*_k = [q_k, \beta_k]$
    end
    $y_i = r_i + \gamma \max_a Q'(q^*_K, x^{\text{static}}_{i+1}, a)$
  end
  perform gradient step and update target network as in Algorithm 1
end

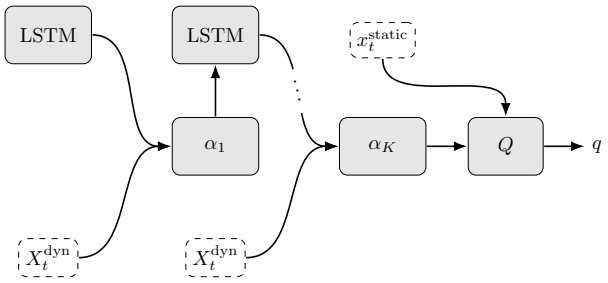


Fig. 3. Unrolled scheme of Set2Set-Q. Dynamic feature representations $X_t^{\text{dyn}} = [x_t^1, .., x_t^{\text{seq len}}]^\top$ are fed via a permutation invariant attention mechanism α_k to the LSTM in K iterations and accumulated in q_k^*. Vector x_t^{static} is the static feature representation at time step t. q is the action-value output.

proposed in [12], which uses an aggregation in a hidden state of a long-term short-term memory (LSTM) [18] as a pooling operation for the input elements. Combined with their proposed attention mechanism, the aggregation of the input set is permutation invariant. The Q-network consists of two main parts (LSTM, Q). An overview is shown in Figure 3. The LSTM is updated K times, where k denotes the iteration index. It evolves a state q_k^* with $q_k = \text{LSTM}(q_{k-1}^*)$, which is formed by a concatenation $q_k^* = [q_k, \beta_k]$ of a query vector q_k with the readout vector $\beta_k = \sum_j \alpha_{j,k} x^j$. The readout vector is computed over all input set elements $x^j \in X^{\text{dyn}}$ multiplied by an attention factor $\alpha_{j,k}(x^j, q_k)$. The resulting state representation of the dynamic objects q_K^* is concatenated with static feature representation x^{static} and fed into the Q-module. For details, see Algorithm 2.

III. Experimental Setup

We apply the Deep Set and Set2Set input representations in the reinforcement learning setting for high-level decision making in autonomous driving.

A. Application to Autonomous Driving

In order to model this task as a MDP, we first define state space, action space and reward function.

1) State Space: For the agent itself, subsequently called ego vehicle, we use the absolute velocity $v_{\text{ego}} \in \mathbb{R}_{\geq 0}$ and whether lanes to the left and right of the agent are available or not. For all vehicles j within the scope of an input representation and within the maximum sensor range d_{max}, we consider the following features:

- the relative distance $dr_j = (p_j - p_{\text{ego}})/d_{\text{max}} \in \mathbb{R}$, where p_{ego}, p_j are longitudinal positions in a curvilinear coordinate system of the lane,
- the relative velocity $dv_j = \frac{v_j - v_{\text{ego}}}{v_{\text{ego}} + \epsilon} \in \mathbb{R}$, where $\epsilon \ll 1$ and v_{ego}, v_j denote absolute velocities,
- and the relative lane[2] $dl_j = l_j - l_{\text{ego}} \in \mathbb{N}$ where l_j, l_{ego} are lane indices.

2) Action Space: The action space consists of a discrete set of possible actions $\mathcal{A} = \{\text{keep lane, perform left lane-change, perform right lane-change}\}$. The actions are high-level decisions of the agent in lateral direction. Acceleration towards reaching the desired velocity is controlled by a low-level execution layer. Collision avoidance and maintaining safe distance to the preceding

[2]e.g. $dl_c = -1$ for vehicle c which is one lane to the left of the agent. This feature is not needed for the fixed input and occupancy grids.

Authorized licensed use limited to: SHANGHAI UNIVERSITY. Downloaded on October 22,2026 at 07:28:47 UTC from IEEE Xplore. Restrictions apply.

vehicle are handled by an integrated safety module, analogously to [11], [19]. In our case the agent simply keeps the lane, if the chosen action is not safe.

3) Reward Function: For a desired velocity of the agent v_{desired}, we define the reward function[3] $r : \mathcal{S} \times \mathcal{A} \mapsto \mathbb{R}$ as:

$$r(s, a) = 1 - \frac{|v_{\text{ego}} - v_{\text{desired}}|}{v_{\text{desired}}} - p_{\text{lc}},$$

where p_{lc} is a penalty for choosing a lane change action. In our experiments, we use $p_{\text{lc}} = 0.01$ if action a is a lane change and 0 otherwise. The weight was chosen empirically in preliminary experiments.

B. Input Representations

In this section, we describe how to transform the given state information to three different input representations. In total, we trained and evaluated four DQN agents with different input modules. For all approaches, including the baselines, we optimized the hyperparameters extensively. A detailed overview of the configuration spaces can be found in the appendix.

1) Relational Grid (Fixed Input): As fixed input representation, we use a relational grid with neighborhood $\Delta_{\text{ahead}} = \Delta_{\text{behind}} = \Delta_{\text{lateral}} = 2$, which results in a maximum of 20 surrounding cars[4]. For non-existent cars in the grid we use default value $dv = 0$, as well as $dr = 1$ for leaders and $dr = -1$ for followers. These values correspond to a vehicle with same speed at maximum sensor range, having no influence on the decisions of the agent. In total, the state consists of 43 input features.

2) Occupancy Grid: A two-dimensional occupancy grid depicts the scene around the ego vehicle from a bird's eye perspective as an input. We use a grid size[5] of 80×5, where the ego vehicle is represented in the middle. We therefore observe 5 highway lanes. All cells in the grid occupied by a surrounding vehicle j are assigned values $1 + dv_j$ and the cells occupied by the ego vehicle $dv_{\text{ego}} = 1$. Free cells are assigned zeros.

3) Set Input: The input for DeepSet-Q and Set2Set-Q consists of a variable length list of feature vectors. If no vehicles are in sensor range, the forward pass of ϕ is omitted for the Deep Sets and a vector of zeros is fed to ρ. In the Set2Set model, the forward passes of the LSTM are skipped and q_K^* is set to zero.

[3]In favor of simplicity, we omit considering more factors such as jerk or cooperativeness.

[4]In preliminary experiments we further investigated a relational grid with $\Delta_{\text{ahead}} = \Delta_{\text{behind}} = \Delta_{\text{lateral}} = 3$, resulting in 42 considered vehicles. However, the results showed very high variance. Additionally, we omitted experiments with 6 vehicles because of the limitations shown in Figure 1.

[5]We also evaluated a grid size of 160×5. In our experiments, however, the larger grid led to a worse performance after the same number of gradient steps.

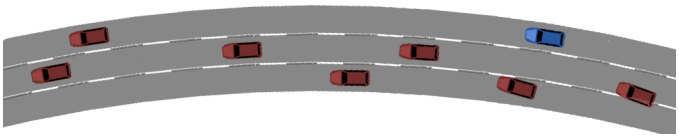


Fig. 4. Birdview of the 3-lane highway scenario in SUMO. The ego vehicle is shown in blue. All other vehicles are shown in red.

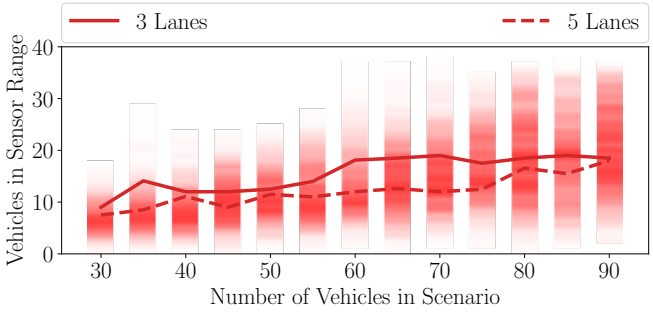


Fig. 5. The vertical bars denote the distribution of transitions for a given number of vehicles in sensor range, shown for scenarios with 30 to 90 vehicles on three lanes. The darker, the higher the frequency. Lines represent the mean number of vehicles in sensor range for three lanes (solid) and five lanes (dashed).

C. SUMO Simulator

For experiments, we used the open-source SUMO traffic simulation framework [20], as shown in Figure 4. Training and evaluation scenarios were performed on a simulated 1000 m circular highway with three lanes. The vehicles were randomly positioned passenger cars with different driving behaviors to resemble realistic highway traffic as much as possible. We set the sensor range to 80 m in front and behind the ego vehicle. Details for the simulation environment and driving behaviors are shown in appendix, Table III.

D. Training Setup

The data was collected in SUMO simulation scenarios. We trained our agents offline on two separate datasets:

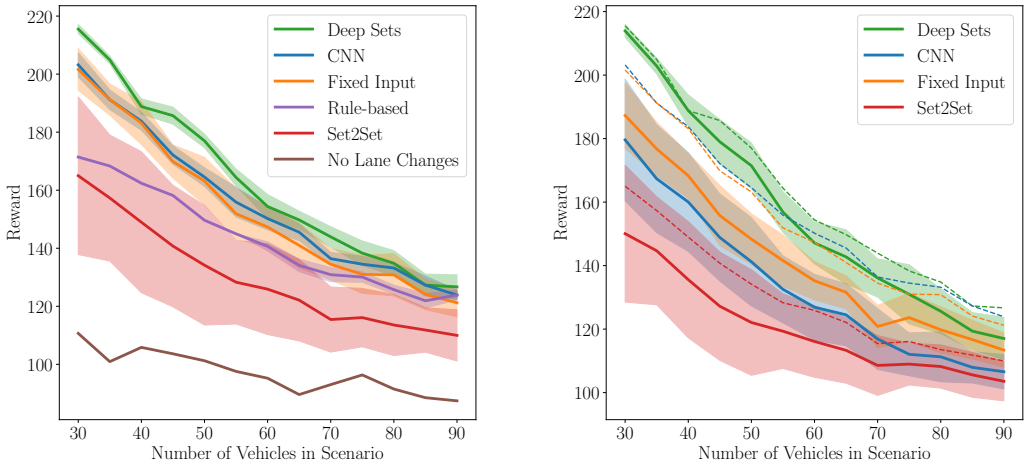
- Dataset 1: 500.000 transition samples with an arbitrary number of surrounding vehicles in each transition.
- Dataset 2: 500.000 transition samples with at most six surrounding vehicles. This dataset was sampled proportional to the original data distribution.

The datasets were collected by a data-collection agent in traffic scenarios with a random number of $n \in [30, 60]$ vehicles. The data-collection agent had the SUMO safety module integrated and performed a random lane change to the left or right whenever possible.

E. Evaluation Setup

Since high-level decision making for lane changes on highways is a very stochastic task – the behaviour and mutual influence of the other drivers is highly unpredictable – we evaluated our agents on a variety of different scenarios, to smooth out the high variance. We generated both three- and five-lane scenarios, varying in the number of vehicles

Authorized licensed use limited to: SHANGHAI UNIVERSITY. Downloaded on October 22,2026 at 07:28:47 UTC from IEEE Xplore. Restrictions apply.



(a) (b)

Fig. 6. (a) Mean performance and standard deviation over 10 training runs on Dataset 1. (b) Mean performance and standard deviation over 10 training runs, trained on Dataset 2 (solid) and on Dataset 1 (dashed).

Scenario	30	50	70	90
Fixed Input	201.59 (± 7.23)	163.17 (± 8.17)	134.47 (± 5.45)	121.21 (± 4.83)
CNN	203.2 (± 4.15)	164.54 (± 3.2)	136.41 (± 2.15)	123.9 (± 1.7)
Set2Set	164.99 (± 27.17)	134.15 (± 20.61)	115.43 (± 11.22)	109.98 (± 8.85)
Deep Sets	**215.51** (± 1.58)	**177.01** (± 2.35)	**143.93** (± 3.52)	**126.7** (± 4.14)

TABLE I

MEAN PERFORMANCE AND STANDARD DEVIATION OF THE APPROACHES TRAINED ON SAMPLES WITH ALL VEHICLES (DATASET 1).

Scenario	30	50	70	90
Fixed Input	187.26 (± 10.33)	148.33 (± 8.0)	120.83 (± 6.64)	113.4 (± 5.4)
Set2Set	150.06 (± 21.55)	122.09 (± 16.64)	108.56 (± 9.43)	103.56 (± 6.15)
CNN	179.6 (± 19.18)	141.21 (± 13.93)	116.89 (± 9.59)	106.57 (± 5.41)
Deep Sets	**213.91** (± 2.02)	**171.46** (± 7.17)	**136.11** (± 5.99)	**117.05** (± 6.69)

TABLE II

MEAN PERFORMANCE AND STANDARD DEVIATION OF THE APPROACHES TRAINED ON SAMPLES WITH UP TO 6 VEHICLES (DATASET 2).

$n \in \{30+i\cdot 5 | 0 \leq i \leq 12\}$. For each fixed n, we evaluated 20 scenarios with different a priori randomly sampled positions and driving behaviours for each vehicle. In total, each agent was evaluated on the same 260 scenarios per fixed lane setting. The distribution of surrounding vehicles for the evaluation scenarios is shown for three and five lanes in Figure 5.

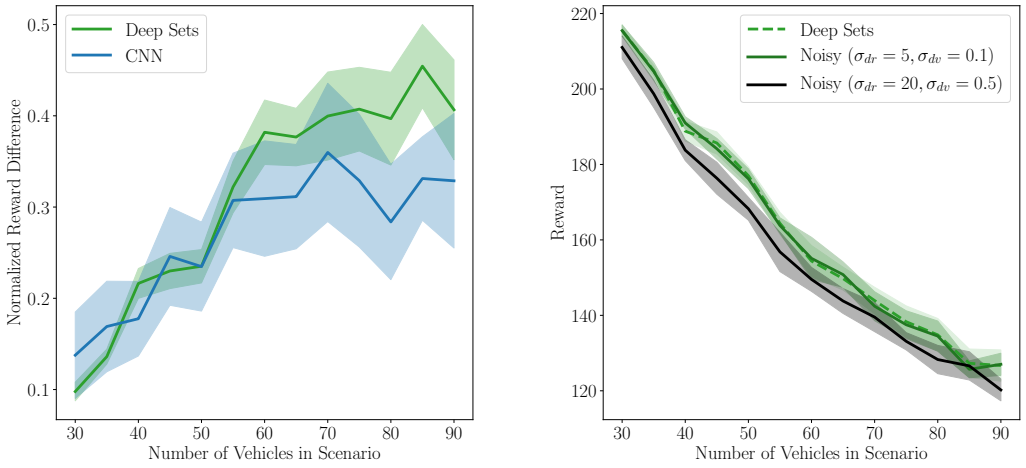
F. Comparative Analysis

DeepSet-Q and Set2Set-Q were compared to a fully-connected neural network using the fixed relational grid input and a convolutional neural network using the occupancy grid. All dynamic input architectures are shown in the appendix, Table V. Additionally, we compared a naive agent with no lane changes and a rule-based controller, that uses the SUMO lane change model *LC2013* with parameters shown in appendix. Each network was trained with a batch size of 64. For optimization of all architectures, we used Adam [21] with a learning rate of 10^{-4}. The target networks were updated with $\tau = 10^{-4}$. Rectified Linear Units (ReLu) were used in all hidden layers of all architectures. The input layers of the network architectures were optimized using Random Search with the same fixed budget for the different approaches. We preferred Random Search over Grid Search because less computational budget is needed to find models with better performance [22]. The final dense layers of the Q-module were optimized once for the fixed input architecture and kept for all other architectures. For the CNN architecture, we used an all-convolutional network [23]. The configuration spaces for the Random Search are shown in the appendix, Table IV.

IV. RESULTS

As can be seen in Figure 6a, the Deep Set architecture yields the best performance and lowest variance across traffic scenarios in comparison to rule-based agents and

Authorized licensed use limited to: SHANGHAI UNIVERSITY. Downloaded on October 22,2026 at 07:28:47 UTC from IEEE Xplore. Restrictions apply.



(a) (b)

Fig. 7. (a) Mean performance and standard deviation for agents trained on scenarios with three lanes and evaluated on scenarios with five lanes. (b) Mean performance and standard deviation for DeepSet-Q agents, evaluated on noisy perception with two different noise levels.

reinforcement learning agents using Set2Set, fully-connected or convolutional neural networks as input modules. For all results, see Table I. A video can be found in the supplementary. The differences between the approaches get smaller as more vehicles are on the track due to general maneuvering limitations in dense traffic.

To investigate the generalization capabilities of all methods we trained additionally on Dataset 2, containing only transitions of at most six surrounding vehicles. As depicted in Figure 5, this is only a small fraction of possible traffic situations. The evaluation results for the truncated dataset are presented in Figure 6b. CNNs have difficulties generalizing to unseen situations and suffer from larger variance in comparison to training on Dataset 1. In contrast, Deep Sets are able to mostly keep the performance even when trained on the truncated dataset, showing only a small increase in variance. The results can be found in Table II.

Results of generalization to a differing number of lanes can be seen in Figure 7a. The plot shows the relative change in performance after increasing the number of lanes from three to five. In these situations, the CNN cannot necessarily represent the outermost lane if the agent is to the full left or right. We hypothesize this the reason for the worse performance in comparison to the Deep Sets for scenarios with a lot of vehicles on the track. In these situations also the outermost lanes can be of high importance due to the crowded traffic, but more analysis is needed to fully elucidate these differences.

As an initial evaluation of transferability of the DeepSet-Q agent to real-world applications, we added Gaussian noise to the relative distance dr and relative velocity dv of all vehicles within sensor range, assuming that the relative lane can be assigned accurately. The results for two different noise levels are depicted in Figure 7b. Our agents show little performance decrease and seem to be robust against reasonable noise levels.

V. Conclusion

In this paper, we evaluated the DeepSet-Q architecture on the problem of high-level decision making in autonomous lane change maneuvers and put the results into context of current research. Deep Sets were able to outperform convolutional neural networks and recurrent attention approaches and demonstrated better generalization to unseen scenarios. Even though recent results present a connection between the bottleneck of embedding $\phi(\cdot)$ and the maximum set size [24], we showed that Deep Sets can offer a both scalable and well-performing alternative to established approaches in the area of dynamic inputs.

Going forward, we believe there lies great potential in extending the architecture to multiple types of objects, such as pedestrians and cyclists, traffic lights or indicators for ending lanes. Further, the temporal context for traffic participants could be incorporated by including recurrent units in ϕ. Finally, the evaluation of the architecture on a real physical system yields an important direction for future work.

VI. Acknowledgment

We would like to thank Manuel Watter for fruitful discussions concerning dynamic input representations.

VII. Appendix

A. SUMO Configuration

SUMO was used with a time step length of $0.5\,\mathrm{s}$ and a lane change duration of $2\,\mathrm{s}$. The action step length for the reinforcement learning agent is $2\,\mathrm{s}$. Acceleration and deceleration of all vehicles are $2.6\,\mathrm{m/s^2}$ and $4.5\,\mathrm{m/s^2}$. The minimum gap is $2\,\mathrm{m}$, the desired time headway to $\tau = 0.5\,\mathrm{s}$. As lane change controller, we use *LC2013* with lcKeepRight $= 0.0$. All car lengths are $4.5\,\mathrm{m}$.

Table III shows the driving behaviors used to create a realistic traffic flow. Driving behaviors can be influenced by the maximum speed, the eagerness for performing lane

Authorized licensed use limited to: SHANGHAI UNIVERSITY. Downloaded on October 22,2026 at 07:28:47 UTC from IEEE Xplore. Restrictions apply.

Driver Type	maxSpeed	lcSpeedGain	lcCooperative
Ego Driver	24	-	-
Driver Type 1	24 + u	i	0.0
Driver Type 2	12 + u	i	1.0
Driver Type 3	18 + u	i	0.8
Driver Type 4	21 + u	i	0.4

TABLE III

SUMO PARAMETERS FOR DIFFERENT DRIVER TYPES, WHERE $u \sim \mathcal{U}(-5, 5)$ AND $i \sim \mathcal{U}(10, 20)$.

changes to gain speed and the willingness to cooperate. For evaluation scenarios, we sampled a priori 100 different drivers uniformly from Type 1-4.

B. Hyperparameter Optimization

All architectures were optimized by Random Search. The corresponding configuration spaces are shown in Table IV. We sampled 20 configurations, where we jointly optimized parameters and architecture layouts.

Architecture	Parameter	Configuration Space
Fixed	Dense($\cdot$)	$50, 100, 200$
	num layers	$2, 3$
Deep Set	ϕ: num layers	$1, 2, 3$
	ϕ: hidden/output dim	$5, 20, 100$
	ρ: num layers	$1, 2, 3$
	ρ: hidden/output dim	$5, 20, 100$
Set2Set	LSTM: num layers	$1, 2$
	Dense($\cdot$)	$32, 64, 100$
	iterations T	$5, 20, 40$
CNN	CONV: num layers	$2, 3$
	kernel sizes	$([7, 3, 2], [2, 1])$
	strides	$([2, 1], [2, 1])$
	filters	$8, 16, 32$

TABLE IV

CONFIGURATION SPACES CONSIDERED IN RANDOM SEARCH FOR THE DEEP SET ARCHITECTURE AND ALL BASELINES.

C. Network Architectures

The hyperparameter-optimized architectures used in Section IV are shown in Table V.

CNN	Set2Set	Deep Sets
Input($B \times 80 \times 5$)	Input($B \times$ seq len $\times 3$)	
$16 \times$ Conv2D(3×1)	LSTM(6)	ϕ: Dense(20), Dense(80)
$32 \times$ Conv2D(3×1)	Dense(32)	ρ: Dense(80), Dense(20)
concat($\cdot$, Input($B \times 3$))		
Dense(100)		
Dense(100)		
Linear(3)		

TABLE V

NETWORK ARCHITECTURES. THE CNN USES STRIDES OF (2×1) AND ZERO PADDING. FOR THE RELATIONAL GRID (NOT SHOWN), INPUT($B \times 43$) IS FED DIRECTLY TO THE DENSE(100) LAYERS.

Algorithm 3: DeepSet-PPO

initialize $\pi_{\mathcal{DS}} = (\phi, \rho, \pi)$ and $\pi^{old}_{\mathcal{DS}} = (\phi^{\text{old}}, \rho^{\text{old}}, \pi^{\text{old}})$
initialize $V_{\mathcal{DS}}$
for *optimization step o=1,2,...* **do**
  run policy $\pi^{\text{old}}_{\mathcal{DS}}$ for e episodes
  compute advantage estimates $\hat{A}_t$
  for *iteration k=1,2,..., K* **do**
    train $V_{\mathcal{DS}}$
    $R_t = \frac{\pi_{\mathcal{DS}}(a_t|s_t)}{\pi^{\text{old}}_{\mathcal{DS}}(a_t|s_t)}$
    $R_t^{\text{CLIP}} = \text{clip}(R_t, 1-\epsilon, 1+\epsilon)$
    optimize surrogate:

$$L^{\text{CLIP}}(\theta^{\pi_{\mathcal{DS}}}) = \hat{\mathbf{E}}_t \left[\min \left(R_t \hat{A}_t, R_t^{\text{CLIP}} \hat{A}_t \right) \right]$$

  end
end

D. PPO

In preliminary experiments, we employed the Deep Set input in Proximal Policy Optimization (PPO) [25]. For details, see Algorithm 3. However, due to the higher demand for training and the non-trivial application to autonomous driving tasks of on-policy algorithms, we switched to off-policy Q-learning. The performance of DeepSet-PPO is depicted in Figure 8.

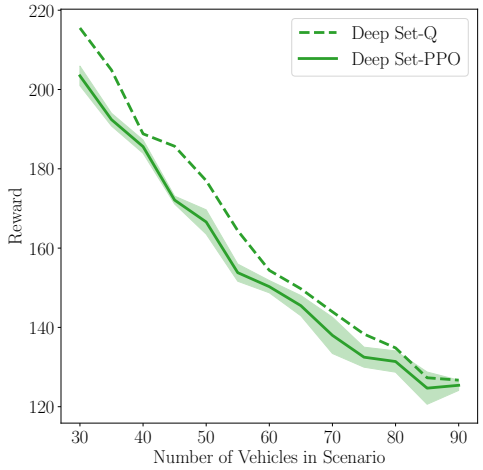


Fig. 8. Mean performance and standard deviation over 7 training runs of DeepSet-PPO, compared to off-policy DeepSet-Q.

The discount factor γ was set to 0.9, since convergence to a satisfying policy needed too many updates, otherwise. We used the Clipped Surrogate Objective, with clipping threshold $\epsilon = 0.2$. Value-function and policy were optimized by Adam with a learning rate of $5 \cdot 10^{-4}$. The Monte Carlo rollout had a horizon of 20. The batch size was set to 64.

Authorized licensed use limited to: SHANGHAI UNIVERSITY. Downloaded on October 22,2026 at 07:28:47 UTC from IEEE Xplore. Restrictions apply.

REFERENCES

[1] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. A. Riedmiller, A. Fidjeland, G. Ostrovski, S. Petersen, C. Beattie, A. Sadik, I. Antonoglou, H. King, D. Kumaran, D. Wierstra, S. Legg, and D. Hassabis, "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, 2015. [Online]. Available: https://doi.org/10.1038/nature14236

[2] D. Silver, A. Huang, C. J. Maddison, A. Guez, L. Sifre, G. van den Driessche, J. Schrittwieser, I. Antonoglou, V. Panneershelvam, M. Lanctot, S. Dieleman, D. Grewe, J. Nham, N. Kalchbrenner, I. Sutskever, T. P. Lillicrap, M. Leach, K. Kavukcuoglu, T. Graepel, and D. Hassabis, "Mastering the game of go with deep neural networks and tree search," *Nature*, vol. 529, no. 7587, pp. 484–489, 2016. [Online]. Available: https://doi.org/10.1038/nature16961

[3] M. Watter, J. T. Springenberg, J. Boedecker, and M. A. Riedmiller, "Embed to control: A locally linear latent dynamics model for control from raw images," in *Advances in Neural Information Processing Systems 28: Annual Conference on Neural Information Processing Systems 2015, December 7-12, 2015, Montreal, Quebec, Canada*, 2015, pp. 2746–2754. [Online]. Available: http://papers.nips.cc/paper/5964-embed-to-control-a-locally-linear-latent-dynamics-model-for-control-from-raw-images

[4] S. Levine, C. Finn, T. Darrell, and P. Abbeel, "End-to-end training of deep visuomotor policies," *Journal of Machine Learning Research*, vol. 17, pp. 39:1–39:40, 2016. [Online]. Available: http://jmlr.org/papers/v17/15-522.html

[5] M. Jaritz, R. de Charette, M. Toromanoff, E. Perot, and F. Nashashibi, "End-to-end race driving with deep reinforcement learning," in *2018 IEEE International Conference on Robotics and Automation, ICRA 2018, Brisbane, Australia, May 21-25, 2018*. IEEE, 2018, pp. 2070–2075. [Online]. Available: https://doi.org/10.1109/ICRA.2018.8460934

[6] P. Wolf, K. Kurzer, T. Wingert, F. Kuhnt, and J. M. Zöllner, "Adaptive behavior generation for autonomous driving using deep reinforcement learning with compact semantic states," *CoRR*, vol. abs/1809.03214, 2018. [Online]. Available: http://arxiv.org/abs/1809.03214

[7] B. Mirchevska, M. Blum, L. Louis, J. Boedecker, and M. Werling, "Reinforcement learning for autonomous maneuvering in highway scenarios." *11. Workshop Fahrerassistenzsysteme und automatisiertes Fahren*.

[8] M. Nosrati, E. A. Abolfathi, M. Elmahgiubi, P. Yadmellat, J. Luo, Y. Zhang, H. Yao, H. Zhang, and A. Jamil, "Towards practical hierarchical reinforcement learning for multi-lane autonomous driving," *2018 NIPS MLITS Workshop*, 2018.

[9] M. Kaushik, V. Prasad, M. Krishna, and B. Ravindran, "Overtaking maneuvers in simulated highway driving using deep reinforcement learning," 06 2018, pp. 1885–1890.

[10] M. Mukadam, A. Cosgun, and K. Fujimura, "Tactical decision making for lane changing with deep reinforcement learning," *NIPS Workshop on Machine Learning for Intelligent Transportation Systems*, 2017.

[11] L. Fridman, B. Jenik, and J. Terwilliger, "DeepTraffic: Driving Fast through Dense Traffic with Deep Reinforcement Learning," *arXiv e-prints*, p. arXiv:1801.02805, Jan. 2018.

[12] O. Vinyals, S. Bengio, and M. Kudlur, "Order Matters: Sequence to sequence for sets," *arXiv e-prints*, p. arXiv:1511.06391, Nov. 2015.

[13] R. Pascanu, T. Mikolov, and Y. Bengio, "On the difficulty of training recurrent neural networks," in *Proceedings of the 30th International Conference on International Conference on Machine Learning - Volume 28*, ser. ICML'13. JMLR.org, 2013, pp. III–1310–III–1318. [Online]. Available: http://dl.acm.org/citation.cfm?id=3042817.3043083

[14] M. Zaheer, S. Kottur, S. Ravanbakhsh, B. Poczos, R. R. Salakhutdinov, and A. J. Smola, "Deep sets," in *Advances in Neural Information Processing Systems 30*, I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, Eds. Curran Associates, Inc., 2017, pp. 3391–3401. [Online]. Available: http://papers.nips.cc/paper/6931-deep-sets.pdf

[15] R. Lee, S. Mou, V. Dasagi, J. Bruce, J. Leitner, and N. Sünderhauf, "Zero-shot sim-to-real transfer with modular priors," *CoRR*, vol. abs/1809.07480, 2018. [Online]. Available: http://arxiv.org/abs/1809.07480

[16] C. J. C. H. Watkins and P. Dayan, "Q-learning," in *Machine Learning*, 1992, pp. 279–292.

[17] S. Fujimoto, H. van Hoof, and D. Meger, "Addressing function approximation error in actor-critic methods," in *Proceedings of the 35th International Conference on Machine Learning, ICML 2018, Stockholmsmässan, Stockholm, Sweden, July 10-15, 2018*, 2018, pp. 1582–1591. [Online]. Available: http://proceedings.mlr.press/v80/fujimoto18a.html

[18] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Comput.*, vol. 9, no. 8, pp. 1735–1780, Nov. 1997. [Online]. Available: http://dx.doi.org/10.1162/neco.1997.9.8.1735

[19] B. Mirchevska, C. Pek, M. Werling, M. Althoff, and J. Boedecker, "High-level decision making for safe and reasonable autonomous lane changing using reinforcement learning," *2018 21st International Conference on Intelligent Transportation Systems (ITSC)*, pp. 2156–2162, 2018.

[20] D. Krajzewicz, J. Erdmann, M. Behrisch, and L. Bieker-Walz, "Recent development and applications of sumo - simulation of urban mobility," *International Journal On Advances in Systems and Measurements*, vol. 3&4, 12 2012.

[21] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," *CoRR*, vol. abs/1412.6980, 2014. [Online]. Available: http://arxiv.org/abs/1412.6980

[22] J. Bergstra and Y. Bengio, "Random search for hyper-parameter optimization," *J. Mach. Learn. Res.*, vol. 13, pp. 281–305, Feb. 2012. [Online]. Available: http://dl.acm.org/citation.cfm?id=2188385.2188395

[23] J. Springenberg, A. Dosovitskiy, T. Brox, and M. Riedmiller, "Striving for simplicity: The all convolutional net," in *ICLR (workshop track)*, 2015. [Online]. Available: http://lmb.informatik.uni-freiburg.de/Publications/2015/DB15a

[24] E. Wagstaff, F. B. Fuchs, M. Engelcke, I. Posner, and M. Osborne, "On the Limitations of Representing Functions on Sets," *arXiv e-prints*, p. arXiv:1901.09006, Jan 2019.

[25] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *CoRR*, vol. abs/1707.06347, 2017. [Online]. Available: http://arxiv.org/abs/1707.06347

Authorized licensed use limited to: SHANGHAI UNIVERSITY. Downloaded on October 22,2026 at 07:28:47 UTC from IEEE Xplore. Restrictions apply.